

Chapter 5 (Process API) — Homework (Code)

Zhiyuan Pan

Code: `q1.c` ... `q8.c` in this folder. Build with `make`, then run `./q1` ... `./q8`. All outputs below come from Ubuntu 24.04 (aarch64, gcc 13.3), run in a terminal. The simulation part of this chapter is in `chapter5.pdf`; to reproduce those runs, use `./simulation.sh`.

Q1

Write a program that calls `fork()`. Before calling `fork()`, have the main process access a variable (e.g., `x`) and set its value to something (e.g., 100). What value is the variable in the child process? What happens to the variable when both the child and parent change the value of `x`?

`q1.c`

```
before fork: x = 100 (pid 63438)
parent: set x = 300, &x = 0xfffffe167c890
child:  x = 100 (inherited)
child:  set x = 200, &x = 0xfffffe167c890
parent: after child exits, x = 300
```

In the child, `x` is **100**, a copy of the parent's value at the time of `fork()`. After that, parent and child have **separate copies**. The child sets it to 200 and the parent sets it to 300, and neither change affects the other; the parent still sees 300 after the child exits.

`&x` prints the same address in both processes because each process has its own virtual address space. The same virtual address maps to different physical memory (Linux uses copy-on-write: the page is copied when one of them first writes to it).

The program calls `fflush(stdout)` before `fork()`. Without it, when `stdout` is a pipe or a file (fully buffered), "before fork" is still in the stdio buffer when `fork()` copies the process. It then gets printed **twice**, once by each process. I saw exactly this when running the program with its output redirected.

Q2

Write a program that opens a file (with the `open()` system call) and then calls `fork()` to create a new process. Can both the child and parent access the file descriptor returned by `open()`? What happens when they are writing to the file concurrently, i.e., at the same time?

`q2.c`

```

file offset after both wrote: 140
$ cat q2.output
parent line 0
parent line 1
parent line 2
parent line 3
parent line 4
child line 0
child line 1
child line 2
child line 3
child line 4

```

Yes, both can use it. `fork()` copies the parent's file descriptor table, so the child also has the descriptor. Both copies point to the **same open file description** in the kernel, which holds the file offset. They therefore **share one offset**: each `write()` lands after the other's data, and nothing is overwritten. After 10 lines $\times$ 14 bytes, the offset is 140. If each process had its own offset, both would start writing at 0, and one would overwrite the other.

When they write concurrently, no data is lost, but the order of lines between the two processes is up to the scheduler. With only 5 lines each, the parent happened to finish before the child started.

Changing `LINES` to 1000 shows real interleaving:

```

file offset after both wrote: 31780
$ awk '{print $1}' q2.output | uniq -c | head
    210 parent
      1 child
      1 parent
      1 child
      1 parent
      ...
$ wc -l q2.output
2000 q2.output

```

The parent wrote 210 lines on its own, then the two processes alternated line by line, yet all 2000 lines are intact. Each line is one small `write()` call, and the kernel updates the shared offset atomically for each call.

Q3

Write another program using `fork()`. The child process should print "hello"; the parent process should print "goodbye". You should try to ensure that the child process always prints first; can

you do this without calling `wait()` in the parent?

q3.c

```
hello
goodbye
```

Yes. The parent creates a `pipe()` before forking and then blocks in `read()` on the pipe. The child prints "hello" and then writes one byte into the pipe. Only then does the parent's `read()` return, and the parent prints "goodbye". The order is guaranteed no matter how the scheduler runs the two processes.

Calling `sleep()` in the parent usually works too, but does **not** guarantee the order. Other correct options are signals (`kill()` and `pause()`) or a shared semaphore.

Q4

Write a program that calls `fork()` and then calls some form of `exec()` to run the program `/bin/ls`. See if you can try all of the variants of `exec()`, including (on Linux) `execl()`, `execle()`, `execlp()`, `execv()`, `execvp()`, and `execvpe()`. Why do you think there are so many variants of the same basic call?

q4.c

```
execl   : drwxrwxrwt 18 root root 4096 Oct  8 13:50 /tmp
execle  : drwxrwxrwt 18 root root 4096 Oct  8 13:50 /tmp
execlp  : drwxrwxrwt 18 root root 4096 Oct  8 13:50 /tmp
execv   : drwxrwxrwt 18 root root 4096 Oct  8 13:50 /tmp
execvp  : drwxrwxrwt 18 root root 4096 Oct  8 13:50 /tmp
execvpe : drwxrwxrwt 18 root root 4096 Oct  8 13:50 /tmp
```

For each variant, the program forks a child, which `execs ls -ld /tmp`, and waits for it. All six work and give the same output. They are library front-ends to one system call, `execve()`, and differ in three independent choices encoded in the letters of the name:

letter	meaning
<code>l</code> (list)	arguments passed one by one: <code>execl("/bin/ls", "ls", "-ld", "/tmp", NULL)</code>
<code>v</code> (vector)	arguments passed as an array: <code>execv("/bin/ls", args)</code>
<code>p</code> (path)	search <code>\$PATH</code> for the program, so <code>"ls"</code> is enough instead of <code>"/bin/ls"</code>
<code>e</code> (environment)	pass an explicit environment array instead of inheriting the current one

There are so many variants for convenience. `l` is easiest when the arguments are fixed when you write the code, and `v` when they are built at runtime (for example by a shell parsing a command line). `p` behaves like typing a command in a shell. `e` lets the caller control the new program's environment, which is useful for security or for setting up a clean environment.

Q5

Now write a program that uses `wait()` to wait for the child process to finish in the parent. What does `wait()` return? What happens if you use `wait()` in the child?

q5.c

```
child (pid 63457): wait() returned -1, errno = No child processes
parent (pid 63456): fork() returned 63457, wait() returned 63457
parent: child exit status = 42
```

In the parent, `wait()` blocks until a child exits and **returns that child's PID**, the same value `fork()` returned. Through its `status` argument it also reports how the child ended: `WEXITSTATUS(status)` is 42, the value the child passed to `exit()`.

In the child, which has no children of its own, `wait()` returns **-1** immediately and sets `errno` to `ECHILD` ("No child processes").

Q6

Write a slight modification of the previous program, this time using `waitpid()` instead of `wait()`. When would `waitpid()` be useful?

q6.c

```
parent: waitpid(slow, WNOHANG) returned 0 (still running)
fast child (pid 63461) exiting
slow child (pid 63460) exiting
parent: waitpid(slow) returned 63460, exit status 1
parent: waitpid(fast) returned 63461, exit status 2
```

`waitpid(pid, &status, options)` waits for one **specific** child. The program starts a slow child (sleeps 2 s) and a fast child. The fast one exits first, but the parent collects the slow one first because it asked for that PID; `wait()` would have returned the fast child. With the `WNOHANG` option `waitpid()` does not block at all: it returns 0 because the slow child is still running.

`waitpid()` is useful when:

- a process has several children and needs a particular one, e.g. a shell waiting for its foreground job while background jobs keep running;
- it wants to check on children without blocking (`WNOHANG`), e.g. to reap finished background jobs between other work;
- it needs to notice children that were stopped or continued (`WUNTRACED`, `WCONTINUED`), as job control in a shell does.

Q7

Write a program that creates a child process, and then in the child closes standard output (`STDOUT_FILENO`). What happens if the child calls `printf()` to print some output after closing the descriptor?

q7.c

```
parent: before fork
child (stderr): printf returned -1, fflush returned 0 (Bad file descriptor)
parent: stdout still works
```

The child's `printf()` output **never appears**, and the program does not crash. `printf()` writes into the stdio buffer, and when the buffer is flushed, the underlying `write(1, ...)` fails with `EBADF` (bad file descriptor) because descriptor 1 is closed. The text is simply discarded. The error is only visible through return values, which the child reports on `stderr` (descriptor 2 is still open):

- In a terminal, `stdout` is line-buffered, so the `\n` makes `printf()` try to write immediately, and `printf()` itself returns -1.
- When `stdout` is a pipe or file (fully buffered), `printf()` returns 23 (the bytes it buffered), and the failure shows up later, when `fflush()` returns -1.

Closing the descriptor in the child does not affect the parent, because each process has its own file descriptor table. The parent's `stdout` still works.

Q8

Write a program that creates two children, and connects the standard output of one to the standard input of the other, using the `pipe()` system call.

q8.c

```
$ ./q8
24
$ ls -l / | wc -l
24
```

The parent creates a pipe and then forks two children:

- **Child 1** calls `dup2(p[1], STDOUT_FILENO)` and then `exec s ls -l /`, so its standard output goes **into** the pipe.
- **Child 2** calls `dup2(p[0], STDIN_FILENO)` and then `exec s wc -l`, so its standard input comes **from** the pipe.

The result is identical to the shell pipeline `ls -l / | wc -l`; this is how a shell implements `|`. Every process must close the pipe ends it does not use, and the parent closes both. If any copy of the write end stayed open, `wc` would never see end-of-file and would wait forever.